Secure Employee Management System

Code Explanation • Setup Guide • API Flow • Security Notes

LAB 5 — Express 5 + TypeScript + PostgreSQL + TypeORM

Beginner-friendly companion guide for the project ZIP

Use this document alongside the source code. File names and commands refer to the supplied project archive.

1. What the project does

This is a backend REST API for employee and department management. It adds login and session management, role-based authorization, resource ownership checks, database constraints, validation, security middleware, and privacy-aware responses.

• **Module A:** employee and department CRUD-style endpoints, pagination, filtering, search, validated sorting, profile endpoints, and soft deactivation.

• **Module B:** login, JWT access tokens, refresh-token rotation, logout, session listing/revocation, and password change.

• **Module C:** Admin, HR, Manager, and Employee roles; middleware checks; and service-level checks for employee ownership/reporting scope.

• **Cross-cutting:** Zod validation, TypeORM migrations, seed data, centralized responses/errors, request IDs, rate limits, CORS, Helmet, and audit logging.

Important: this is an educational starter project. The supplied README identifies several extension points and does not claim that every production-grade control is complete.

2. Technology map

Technology	What it does in this project
Node.js + Express 5	Runs the HTTP server and routes requests to handlers.
TypeScript	Adds types and makes class/service boundaries easier to understand.
PostgreSQL	Stores users, employees, departments, roles, permissions, refresh tokens, and audit logs.
TypeORM	Maps entity classes to tables and runs migrations/repository operations.
Zod	Validates environment settings, request bodies, route parameters, and query values.
Argon2id	Hashes passwords before they are stored.
JWT / jsonwebtoken	Signs and verifies short-lived access tokens.
Helmet, CORS, rate limiting	Adds security headers, restricts browser origins, and limits repeated requests.

3. Folder structure — where to look

Path	Responsibility
src/server.ts	Starts the app, connects to PostgreSQL, and closes resources on shutdown.
src/app.ts	Builds Express middleware stack and mounts routes.
src/config/env.ts	Reads and validates environment variables with Zod.
src/config/database.ts	Defines the TypeORM DataSource and entity/migration locations.
src/entities/	TypeORM table mappings such as User, Employee, Department, Role, RefreshToken, AuditLog.
src/migrations/	Versioned schema creation/change scripts; run these instead of synchronize.
src/seeds/seed.ts	Creates development roles, permissions, a department, and a demo admin.
src/routes/	Maps HTTP methods/URLs to middleware and controller methods.
src/middlewares/	Authentication, role/permission checks, validation, request IDs, rate limiting, and error handling.
src/controllers/	Reads validated HTTP input and sends responses.
src/services/	Business rules: authentication, ownership, manager scope, employee/department rules.

Path	Responsibility
src/repositories/	Database access using TypeORM.
src/validators/	Zod schemas for request data.
src/utils/ApiResponse.ts	Consistent success response shape.
src/errors/app-error.ts	Typed application errors with HTTP status codes.
tests/	Vitest tests for selected security and authorization behavior.
employee-management.postman_collection.json	Importable requests for manual API exploration.

4. Request flow — the most important concept

```
Client / Postman
      |
      v
Express app (src/app.ts)
      |
      +--> request ID, Helmet, CORS, JSON-size/content-type checks
      |
      v
Route (src/routes/*.routes.ts)
      |
      v
Middleware
  - validate request with Zod
  - authenticate JWT if endpoint is protected
  - check role/permission when required
      |
      v
Controller class
  - reads params/body/query and calls service
  - sends a consistent response
      |
      v
Service class
  - applies business rules
  - checks ownership / manager reporting scope
  - coordinates multi-step work
      |
      v
Repository class
  - uses TypeORM to read/write records
      |
      v
PostgreSQL (constraints, foreign keys, indexes)
```

If a route or service throws an AppError, Express passes the error to the centralized error middleware. That handler turns it into a predictable JSON error with the request ID. Unknown URLs go to the not-found handler.

Why keep these layers separate?

• Routes answer: *which middleware and controller should run?*

• Middleware answers: *is the request valid and is the caller authenticated/allowed?*

• Controller answers: *what HTTP input was supplied, and what response should be sent?*

• Service answers: *is this operation allowed by the business rules?*

• Repository answers: *how do we read or write the database?*

• Entity/migration answers: *what is the database structure and its integrity rules?*

5. How to install and run it

Prerequisites

• Install a supported Node.js LTS release and npm.

• Install PostgreSQL and ensure its service is running.

• Extract the ZIP, then open the inner **employee-management-api** folder in VS Code/terminal.

Step 1 — install packages

```
npm install
```

Step 2 — create the database

```
psql -U postgres
CREATE DATABASE employee_management;
\q
```

If you use pgAdmin, create a database named employee_management from the Databases menu instead.

Step 3 — create .env

```
# Windows PowerShell
Copy-Item .env.example .env

# macOS / Linux
cp .env.example .env
```

Open .env and set DATABASE_URL and a strong JWT_ACCESS_SECRET (at least 32 characters). Example format below; replace USER and PASSWORD with your local PostgreSQL credentials.

```
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://USER:PASSWORD@localhost:5432/employee_management
JWT_ACCESS_SECRET=replace_with_a_random_secret_at_least_32_chars
JWT_ACCESS_TTL=15m
REFRESH_TOKEN_TTL_DAYS=7
CORS_ORIGINS=http://localhost:5173
RATE_LIMIT_WINDOW_MS=60000
RATE_LIMIT_MAX_REQUESTS=30
AUDIT_RETENTION_DAYS=365
COOKIE_SECURE=false
```

Do not paste real secrets into chat, commit .env, or reuse development secrets in production.

Step 4 — run the migration

```
npm run migration:run
```

The migration creates tables, relationships, constraints, and indexes. Do not manually create the same tables and do not enable TypeORM synchronize for production-like use.

Step 5 — seed development data

```
# PowerShell
$env:SEED_PASSWORD='Use-A-Long-Dev-Password-123'
npm run seed

# macOS / Linux
SEED_PASSWORD='Use-A-Long-Dev-Password-123' npm run seed
```

The seed script refuses production mode. Use only fake development data. Check the seed file and terminal output for the exact demo login email.

Step 6 — start and check

```
npm run dev
```

Open or request **GET http://localhost:3000/health**. A healthy response means the HTTP app started and its database initialization succeeded.

6. Commands to remember

Command	Purpose
npm install	Install project dependencies.
npm run dev	Run TypeScript in development mode with restart-on-change.
npm run build	Compile TypeScript using tsconfig.json.
npm start	Run the compiled server (after a successful build).
npm run migration:run	Apply pending database migrations.
npm run migration:revert	Revert the latest migration; use carefully.
npm run migration:generate	Generate a migration from entity/schema differences; inspect it before running.
npm run seed	Insert development roles, permissions, department, and demo admin.
npm test	Run the Vitest test suite once.
npm run test:watch	Run tests in watch mode.

7. Authentication and token flow

```
Login (email + password)
   |
   v
Look up user and verify Argon2id password hash
   |
   +--> invalid -> generic login error
   |
   v
Create short-lived signed JWT access token
Create random refresh token
Store only refresh-token hash in database
   |
   v
Client sends access token:
Authorization: Bearer <accessToken>
   |
   v
authenticate middleware verifies signature, expiry,
issuer, audience, token type, and active user/session rules
```

Refresh flow: the client sends the refresh token to POST /api/v1/auth/refresh. The server hashes it, checks it, revokes/rotates the old token and returns a new token pair. Reusing a previously rotated token should be rejected and the affected token family is intended to be revoked.

This educational project sends refresh tokens in JSON. A browser client must not store them in localStorage in production. A safer browser design commonly uses HttpOnly, Secure, SameSite cookies plus CSRF protection where applicable.

8. Roles and ownership

Role	Intended access
Admin	Full employee/department administration and privileged account/security operations.
HR	Create/update employee records and manage departments, but cannot grant itself admin privileges.
Manager	Access permitted employee information within their reporting scope; no unrelated restricted records.
Employee	View own profile and update only approved fields such as first/last name, phone, and job title.

Authentication answers **who are you?** Authorization answers **what may you do?** Ownership checks answer **is this particular record within your permitted scope?** All three matter.

Attack demonstration: send PATCH /api/v1/employees/me with {"role":"admin"}. Strict Zod validation should reject the unknown field. Then try GET /api/v1/employees/<another-user-id> as an employee; the service should deny access without exposing the other person's private record. Finally, reuse an old refresh token after rotation; the replay must fail.

9. API endpoint reference

Method + path	Purpose / access
POST /api/v1/auth/login	Login with email and password; rate-limited.
POST /api/v1/auth/refresh	Rotate refresh token.
POST /api/v1/auth/logout	Revoke current refresh-token session; access token required.
POST /api/v1/auth/logout-all	Revoke all refresh sessions for current user.
GET /api/v1/auth/sessions	List current user's sessions.
DELETE /api/v1/auth/sessions/:id	Revoke a selected session.
POST /api/v1/auth/change-password	Change password; existing sessions should be invalidated as implemented.
POST /api/v1/employees	Create employee; admin/HR.
GET /api/v1/employees	Paginated directory; supports filtering/search/sorting within allowlists.
GET /api/v1/employees/:id	Get employee, subject to resource-level authorization.
PATCH /api/v1/employees/:id	Update employee subject to role and reporting scope.
DELETE /api/v1/employees/:id	Deactivate employee rather than hard-delete.
GET /api/v1/employees/me	Get current employee profile.
PATCH /api/v1/employees/me	Update only permitted self-profile fields.
GET /api/v1/employees/me/export	Export own profile as structured JSON.
GET /api/v1/departments	List departments.
POST /api/v1/departments	Create department; admin/HR.
PATCH /api/v1/departments/:id	Update department; admin/HR.

10. Postman: how to test a protected endpoint

• Import **employee-management.postman_collection.json** into Postman.

• Start the server and send the login request using the demo account created by the seed script.

• Copy the returned accessToken. For protected requests, set Authorization type to Bearer Token and paste the access token.

• For POST/PATCH requests, set Content-Type: application/json and send only fields accepted by the endpoint schema.

• Use actual UUIDs returned by the API. Do not copy placeholder IDs literally.

• Test both allowed and denied behavior: unauthenticated request, wrong role, another employee's ID, invalid body fields, invalid sort field, and repeated refresh token.

11. Database design and migrations

The migration creates relational tables for users, employees, departments, roles, permissions, role_permissions, refresh_tokens, and audit_logs. Foreign keys prevent references to nonexistent related rows; unique constraints protect

identifiers such as work email; checks constrain valid values; indexes help common lookups.

• Entity files describe how TypeORM maps TypeScript classes to database tables.

• Migration files are the versioned source of truth for schema changes.

• Seed scripts add reproducible fake development data; they are not production data.

• Use transactions when an operation changes multiple related rows and all changes must succeed together.

• Let database constraints be the final guard against duplicates and invalid relationships; application validation alone is not enough.

12. Centralized responses, errors, and request IDs

Success responses use ApiResponse so endpoints follow a common shape: success, message, data, and requestId. AppError represents expected failures with an HTTP status. The not-found handler handles unknown routes, and the final error middleware converts thrown/forwarded errors into safe JSON.

```
{
  "success": true,
  "message": "Employee fetched successfully",
  "data": { "...": "..." },
  "requestId": "request-uuid"
}
```

Every response should carry a request ID (also exposed in X-Request-Id) so an issue can be correlated without logging passwords, tokens, or full employee records.

13. Security and privacy checklist

• Never return password hashes, refresh-token hashes, JWT secrets, or internal security metadata.

• Use HTTPS in deployed environments; keep secrets out of Git and logs.

• Keep Helmet, explicit CORS allowlist, request-size limits, content-type checks, and rate limits enabled.

• Validate body, params, and query strings. Allowlist sort columns and writable properties.

• Do not trust a user ID or role sent in a request body to establish identity.

• Deactivate accounts and revoke refresh sessions when access should end.

• Collect only necessary personal data; limit who can read sensitive fields and log only justified audit metadata.

• Define and review retention/deletion policy with the organisation. GDPR compliance depends on more than code; legal/privacy review may be needed.

14. Troubleshooting

Problem	What to check
Configuration invalid at startup	Confirm .env exists, DATABASE_URL is valid, and JWT_ACCESS_SECRET is at least 32 characters.
Cannot connect to PostgreSQL	Check PostgreSQL service, host/port, database name, username/password, and network access.
Relation/table does not exist	Run npm run migration:run against the same database named in DATABASE_URL.
Login fails	Run seed in development, verify the exact seeded email/password, and ensure the user is active.
401 Unauthorized	Send a current access token as Authorization: Bearer <token>; check expiry and token type.
403 Forbidden or 404 on another record	The current role/ownership policy may intentionally block the resource.
CORS error	Add the exact frontend origin to CORS_ORIGINS; include scheme and port, separated by commas.
Port already in use	Change PORT in .env or stop the other process using that port.

Problem	What to check
TypeScript build failure	Run npm run build and inspect the first TypeScript error; don't assume a running dev server means the build passes.

15. What to know before demonstrating to your trainer

• Explain one request end-to-end using the route → middleware → controller → service → repository → PostgreSQL flow.

• Show the database migration and explain primary keys, foreign keys, unique constraints, and indexes.

• Demonstrate the three required attacks: privilege escalation, broken object-level authorization (BOLA), and refresh-token replay.

• Run npm test and npm run build. Report failures honestly and fix them before submission.

• Explain why deactivation is used instead of immediate deletion for employee records.

• Explain that short-lived access JWTs may remain valid until expiry unless immediate revocation checks are implemented.

• Explain why the code is privacy-aware but cannot alone guarantee GDPR compliance.

16. Verification status and known limitations

This guide reflects the supplied ZIP's README and source layout. The project was not verified here by a successful dependency installation, TypeScript build, or run against a live PostgreSQL instance. Run those checks locally before submission.

• The README identifies a dedicated admin-only user/role assignment API as an extension point.

• It recommends stronger concurrency-safe refresh rotation with row locks or conditional updates.

• It identifies complete integration tests with a disposable PostgreSQL database and a scheduled audit-retention worker as remaining work.

• Review the manager reporting-scope policy and sensitive-field handling against your trainer's exact expectations.

• Do not deploy with development seed accounts or development secrets.

17. Quick command sheet

```
# Install / configure
npm install
# create database employee_management in PostgreSQL
Copy-Item .env.example .env             # PowerShell
# edit .env before continuing

# Database and development data
npm run migration:run
$env:SEED_PASSWORD='Use-A-Long-Dev-Password-123'; npm run seed

# Run / verify
npm run dev
# GET http://localhost:3000/health
npm test
npm run build

# Other useful commands
npm run migration:revert
npm run migration:generate
npm run test:watch
npm start
```

Use the macOS/Linux equivalents shown in Section 5 when not using PowerShell. The command `npm start` is for the compiled build, so run `npm run build` first.